

Clustering program

1.0

Generated by Doxygen 1.9.8

1 clust.f90	1
1.1 Description	1
1.2 Parser description	1
1.3 Required Arguments	1
1.4 Optional arguments	1
1.4.1 Modules Used	1
1.4.2 Main Routines Called	2
1.4.3 Notes	2
1.4.4 Usage Example	2
1.4.5 Author	2
2 contact_map.f90	3
2.1 Description	3
2.2 Parser description	3
2.3 Required Arguments	3
2.4 Optional arguments	3
2.4.1 Modules Used	3
2.4.2 Main Routines Called	4
2.4.3 Output	4
2.4.4 Usage Example	4
2.4.5 Author	4
3 make_data.f90	5
3.1 Description	5
3.2 Parser description	5
3.3 Required Arguments	5
3.4 Optional arguments	5
3.4.1 Modules Used	6
3.4.2 Usage	6
3.4.3 Author	6
4 threshold.f90	7
4.1 Description	7
4.2 Parser description	7
4.3 Required Arguments	7
4.4 Optional arguments	7
4.4.1 Modules Used	7
4.4.2 Main Routines Called	8
4.4.3 Usage Example	8
4.4.4 Author	8
5 Fortran Unit Tests (<tt>src/unit_test</tt>)	9
5.1 Included Test Programs	9
5.2 Current Coverage Notes	9

5.3 Default Test Execution Paths	10
5.3.1 From <tt>src/</tt> (Makefile target)	10
5.3.2 From <tt>src/unit_test/</tt> (script)	10
5.4 Author	10
6 Modules Index	11
6.1 Modules List	11
7 Data Type Index	13
7.1 Data Types List	13
8 File Index	15
8.1 File List	15
9 Module Documentation	17
9.1 maths Module Reference	17
9.1.1 Function/Subroutine Documentation	17
9.1.1.1 calculate_cog()	17
9.1.1.2 calculate_distance()	18
9.1.1.3 cross()	18
9.1.1.4 update_complex()	18
9.1.1.5 vectors_angle_2d()	19
9.1.1.6 vectors_angle_3d()	19
9.2 mod_array Module Reference	20
9.2.1 Function/Subroutine Documentation	20
9.2.1.1 array_angle()	20
9.2.1.2 array_atoms_dist()	21
9.2.1.3 array_z_coord()	21
9.2.1.4 merge_sorted_segments()	22
9.2.1.5 read_array()	23
9.2.1.6 write_array()	23
9.3 mod_assoc Module Reference	23
9.3.1 Function/Subroutine Documentation	24
9.3.1.1 allocate_assoc_object()	24
9.3.1.2 fill_assoc_object()	24
9.3.1.3 size_assoc()	24
9.3.1.4 write_complexes()	25
9.4 mod_clust_algorithm Module Reference	25
9.4.1 Function/Subroutine Documentation	25
9.4.1.1 linkage_clustering()	25
9.4.1.2 write_clust_info()	26
9.4.1.3 write_cluster_complexes()	26
9.4.1.4 write_cluster_elements()	27
9.5 mod_pdb Module Reference	27

9.5.1 Function/Subroutine Documentation	28
9.5.1.1 allocate_pdb_object()	28
9.5.1.2 count_atoms()	28
9.5.1.3 count_chains()	28
9.5.1.4 count_residues()	29
9.5.1.5 fill_pdb_object()	29
9.6 mod_threshold Module Reference	30
9.6.1 Function/Subroutine Documentation	30
9.6.1.1 array_angle()	30
9.6.1.2 array_atoms_dist()	31
9.6.1.3 array_z_coord()	31
9.6.1.4 sort_array()	32
9.7 read_input Module Reference	32
9.7.1 Function/Subroutine Documentation	32
9.7.1.1 read_assoc()	32
9.7.1.2 read_pdb()	33
10 Data Type Documentation	35
10.1 mod_assoc::type_assoc_file Type Reference	35
10.1.1 Detailed Description	35
10.1.2 Member Data Documentation	35
10.1.2.1 head	35
10.1.2.2 lines	36
10.1.2.3 nlines	36
10.1.2.4 rot1	36
10.1.2.5 rot2	36
10.1.2.6 trans_vector	36
10.1.2.7 xc1	36
10.1.2.8 xc2	36
10.2 mod_pdb::type_pdb_atom Type Reference	37
10.2.1 Member Data Documentation	37
10.2.1.1 beta	37
10.2.1.2 chainid	37
10.2.1.3 coord	37
10.2.1.4 id	37
10.2.1.5 name	37
10.2.1.6 occ	38
10.2.1.7 record	38
10.2.1.8 resid	38
10.2.1.9 resname	38
10.3 mod_pdb::type_pdb_chain Type Reference	38
10.3.1 Member Data Documentation	39

10.3.1.1 atoms	39
10.3.1.2 chainid	39
10.3.1.3 natoms	39
10.3.1.4 nresidues	39
10.3.1.5 residues	39
10.4 mod_pdb::type_pdb_file Type Reference	40
10.4.1 Detailed Description	40
10.4.2 Member Data Documentation	41
10.4.2.1 atoms	41
10.4.2.2 chains	41
10.4.2.3 natoms	41
10.4.2.4 nchains	41
10.4.2.5 nresidues	41
10.4.2.6 residues	41
10.5 mod_pdb::type_pdb_residue Type Reference	41
10.5.1 Member Data Documentation	42
10.5.1.1 atoms	42
10.5.1.2 chainid	42
10.5.1.3 natoms	42
10.5.1.4 resid	42
10.5.1.5 resname	42
11 File Documentation	43
11.1 clust.f90 File Reference	43
11.1.1 Detailed Description	43
11.1.2 Function/Subroutine Documentation	44
11.1.2.1 main()	44
11.1.2.2 parse_arguments()	44
11.2 clust.md File Reference	44
11.3 contact_map.f90 File Reference	44
11.3.1 Detailed Description	45
11.3.2 Function/Subroutine Documentation	45
11.3.2.1 main()	45
11.3.2.2 print_help()	45
11.4 contact_map.md File Reference	45
11.5 make_data.f90 File Reference	45
11.5.1 Detailed Description	46
11.5.2 Function/Subroutine Documentation	46
11.5.2.1 main()	46
11.5.2.2 parse_arguments()	47
11.6 make_data.md File Reference	48
11.7 maths.f90 File Reference	48

11.7.1 Detailed Description	48
11.8 mod_array.f90 File Reference	49
11.8.1 Detailed Description	49
11.9 mod_assoc.f90 File Reference	49
11.9.1 Detailed Description	50
11.10 mod_clust_algorithm.f90 File Reference	50
11.10.1 Detailed Description	51
11.11 mod_pdb.f90 File Reference	51
11.11.1 Detailed Description	52
11.12 mod_threshold.f90 File Reference	52
11.12.1 Detailed Description	53
11.13 read_input.f90 File Reference	53
11.13.1 Detailed Description	53
11.14 threshold.f90 File Reference	54
11.14.1 Detailed Description	54
11.14.2 Function/Subroutine Documentation	54
11.14.2.1 main()	54
11.14.2.2 parse_arguments()	55
11.15 threshold.md File Reference	55
11.16 unit_test/README.md File Reference	55
11.17 unit_test/run_all_tests.sh File Reference	55
11.18 unit_test/test_array.f90 File Reference	55
11.18.1 Function/Subroutine Documentation	55
11.18.1.1 test_array()	55
11.18.1.2 test_write_read_array()	56
11.19 unit_test/test_assoc.f90 File Reference	56
11.19.1 Function/Subroutine Documentation	57
11.19.1.1 test_assoc()	57
11.19.1.2 test_read_assoc_file()	58
11.20 unit_test/test_clustering.f90 File Reference	58
11.20.1 Detailed Description	58
11.20.2 Function/Subroutine Documentation	59
11.20.2.1 test_clustering()	59
11.20.2.2 test_clustering_statistics()	59
11.21 unit_test/test_maths.f90 File Reference	59
11.21.1 Function/Subroutine Documentation	60
11.21.1.1 test_cross_product()	60
11.21.1.2 test_maths()	60
11.22 unit_test/test_pdb.f90 File Reference	61
11.22.1 Function/Subroutine Documentation	62
11.22.1.1 test_pdb()	62
11.22.1.2 test_pdb_allocation()	62

11.23 unit_test/test_read_input.f90 File Reference	63
11.23.1 Function/Subroutine Documentation	63
11.23.1.1 test_read_input()	63
11.23.1.2 test_read_input_assoc()	64
11.24 unit_test/test_threshold.f90 File Reference	65
11.24.1 Function/Subroutine Documentation	65
11.24.1.1 test_array_z_coord()	65
11.24.1.2 test_threshold()	65
Index	67

Chapter 1

clust.f90

1.1 Description

`clust` performs hierarchical clustering on encounter data using mean linkage.

The input passed with `-input` must be a single-line 1D array generated by `make_data`.

The program reads encounter complexes, runs clustering, and writes output files using the provided output name.

1.2 Parser description

1.3 Required Arguments

- `-complexes <file>`
- `-input <file>`
- `-output_name <name>`

1.4 Optional arguments

- `-nb_encounters <N>` (default: all)
- `-help`

1.4.1 Modules Used

- `mod_array`
- `mod_clust_algorithm`
- `read_input`

1.4.2 Main Routines Called

- `read_assoc` (from `read_input`)
- `read_array` (from `mod_array`)
- `linkage_clustering` (from `mod_clust_algorithm`)
- `print_help` (internal)

1.4.3 Notes

- Clustering runs on CPU for array inputs.
- If `-nb_encounters` is omitted, all encounters in the complexes file are used.

1.4.4 Usage Example

```
./clust -complexes assoc_complexes -input array_z.txt -nb_encounters 5000 -output_name Cu_z
```

1.4.5 Author

Abraham Muñoz-Chicharro

Chapter 2

contact_map.f90

2.1 Description

`contact_map` evaluates residue proximity to a surface across encounter complexes.

For each residue in `pdb2`, it computes a center-of-geometry, applies encounter transformations, and records encounters where the transformed residue position is below a distance threshold.

2.2 Parser description

2.3 Required Arguments

- `-pdb2 <file>`
- `-complexes <file>`

2.4 Optional arguments

- `-nb_encounters <N>` (default: all)
- `-threshold <value>` (default: 6.0)
- `-help`

2.4.1 Modules Used

- `read_input`
- `maths`

2.4.2 Main Routines Called

- `read_pdb` (from `read_input`)
- `read_assoc` (from `read_input`)
- `calculate_cog` (from `maths`)
- `update_complex` (from `maths`)
- `print_help` (internal)

2.4.3 Output

- `closest_residues.txt`: each residue line includes encounter indexes that satisfy the threshold condition.

2.4.4 Usage Example

```
./contact_map -pdb2 p2_noh.pdb -complexes assoc_complexes -nb_encounters 5000 -threshold 6.0
```

2.4.5 Author

Abraham Muñoz-Chicharro

Chapter 3

make_data.f90

3.1 Description

`make_data` generates encounter-derived numeric arrays and writes them to disk.

The selected mode is controlled by `-data_type`:

- `z_coord`
- `atoms_dist`
- `2D_angle`
- `3D_angle`

3.2 Parser description

3.3 Required Arguments

- `-pdb2 <file>`
- `-atoms2 <atom ...>` (or `-atoms2a`)
- `-complexes <file>`
- `-data_type <type>`
- `-output <file>`

3.4 Optional arguments

- `-pdb1 <file>` (required for `atoms_dist` and angle types)
- `-atoms1, -atoms1a, -atoms1b, -atoms2b` (as needed by data type)
- `-nb_encounters <N>`
- `-help`

3.4.1 Modules Used

- `read_input`
- `mod_array`
- `mod_assoc`

3.4.2 Usage

```
./make_data -help  
./make_data -data_type z_coord -help  
./make_data -pdb2 p2_noh.pdb -atoms2 Cu -complexes assoc_complexes -data_type z_coord -output array_z.txt
```

3.4.3 Author

Abraham Muñiz-Chicharro

Chapter 4

threshold.f90

4.1 Description

`threshold` filters encounters using a precomputed array generated by `make_data`.

It reads the array and complexes file, applies `-cutoff`, writes a filtered array file, and writes filtered complexes.

4.2 Parser description

4.3 Required Arguments

- `-array_input <file>`
- `-array <file>`
- `-complexes <file>`
- `-complexes_output <file>`
- `-cutoff <value>`

4.4 Optional arguments

- `-nb_encounters <N>`
- `-help`

4.4.1 Modules Used

- `read_input`
- `mod_array` (`read_array`, `write_array`)
- `mod_threshold`

4.4.2 Main Routines Called

- read_assoc (from `read_input`)
- read_array, write_array (from `mod_array`)
- sort_array (from `mod_threshold`)
- write_cutoff_complexes (internal)
- print_help (internal)

4.4.3 Usage Example

```
./threshold -array_input array_z.txt -complexes assoc_complexes -array threshold_z_array.txt  
-complexes_output threshold_z.txt -cutoff 5.0
```

4.4.4 Author

Abraham Muñiz-Chicharro

Chapter 5

Fortran Unit Tests (`src/unit_test`)

This directory contains standalone Fortran test programs for core clustering modules.

5.1 Included Test Programs

- `test_maths.f90`: numerical/geometry operations in `maths`
- `test_array.f90`: array I/O, array-based encounter metrics, and array/metadata reordering in `mod_array`
- `test_threshold.f90`: threshold operations in `mod_threshold`
- `test_pdb.f90`: PDB parsing/structures in `mod_pdb`
- `test_clustering.f90`: clustering logic in `mod_clust_algorithm`
- `test_assoc.f90`: association file handling in `mod_assoc`
- `test_read_input.f90`: integrated input loading in `read_input`

5.2 Current Coverage Notes

- `test_array.f90` currently covers:
 - `write_array()` / `read_array()`
 - `array_z_coord()`
 - `array_atoms_dist()`
 - `array_angle()`
 - `merge_sorted_segments()`
- `test_threshold.f90` covers threshold-oriented array calculations in `mod_threshold`.

5.3 Default Test Execution Paths

5.3.1 From <tt>src/</tt> (Makefile target)

```
make test
```

This is the authoritative default path. It runs the standard Fortran test set defined in `src/Makefile`, including `test_array`.

5.3.2 From <tt>src/unit_test/</tt> (script)

```
bash run_all_tests.sh
```

This script compiles/runs the list currently defined in `run_all_tests.sh`.

5.4 Author

Abraham Muñiz-Chicharro

Chapter 6

Modules Index

6.1 Modules List

Here is a list of all modules with brief descriptions:

maths	17
mod_array	20
mod_assoc	23
mod_clust_algorithm	25
mod_pdb	27
mod_threshold	30
read_input	32

Chapter 7

Data Type Index

7.1 Data Types List

Here are the data types with brief descriptions:

mod_assoc::type_assoc_file	
Data type for storing binned data (eg	35
mod_pdb::type_pdb_atom	37
mod_pdb::type_pdb_chain	38
mod_pdb::type_pdb_file	
Data type for storing binned data (eg	40
mod_pdb::type_pdb_residue	41

Chapter 8

File Index

8.1 File List

Here is a list of all files with brief descriptions:

clust.f90	Hierarchical clustering driver for encounter arrays	43
contact_map.f90	Residue proximity analysis across transformed encounters	44
make_data.f90	Build encounter-based array data from structural inputs	45
maths.f90	Mathematical and geometric utilities for clustering workflows	48
mod_array.f90	Build/read/write encounter-based arrays and per-encounter metrics	49
mod_assoc.f90	Association/complexes file data structures and parsing routines	49
mod_clust_algorithm.f90	Hierarchical clustering algorithms and output helpers	50
mod_pdb.f90	PDB data structures and parsing/allocation routines	51
mod_threshold.f90	Utilities to compute threshold arrays from encounter complexes	52
read_input.f90	Input readers for PDB and association/complexes files	53
threshold.f90	Threshold-based encounter selection from precomputed arrays	54
unit_test/run_all_tests.sh		55
unit_test/test_array.f90		55
unit_test/test_assoc.f90		56
unit_test/test_clustering.f90	Unit tests for clustering algorithms in mod_clust_algorithm	58
unit_test/test_maths.f90		59
unit_test/test_pdb.f90		61
unit_test/test_read_input.f90		63
unit_test/test_threshold.f90		65

Chapter 9

Module Documentation

9.1 maths Module Reference

Functions/Subroutines

- subroutine [cross](#) (v1, v2, v3)
Compute the cross product of two 3D vectors.
- subroutine [update_complex](#) (xc1, xc2, trans, rot_vx, rot_vy, nb_atoms, coords, new_coords)
Apply translation and rotation to a set of atomic coordinates.
- subroutine [vectors_angle_3d](#) (vector1, vector2, theta_degrees)
Compute the angle in degrees between two 3D vectors.
- subroutine [vectors_angle_2d](#) (vector1, vector2, theta_degrees)
Compute the angle in degrees between two vectors projected to 2D.
- subroutine [calculate_cog](#) (cog, coords, nb_atoms)
Compute the center of geometry (centroid) of a set of points.
- subroutine [calculate_distance](#) (p1, p2, dist)
Compute Euclidean distance between two 3D points.

9.1.1 Function/Subroutine Documentation

9.1.1.1 [calculate_cog\(\)](#)

```
subroutine maths::calculate_cog (  
    real (kind=8), dimension(3), intent(out) cog,  
    real (kind=8), dimension(nb_atoms, 3), intent(in) coords,  
    integer, intent(in) nb_atoms )
```

Compute the center of geometry (centroid) of a set of points.

Sums the coordinates and divides by the number of points to produce the centroid.

Parameters

out	<i>cog</i>	Centroid vector (3 elements)
in	<i>coords</i>	Input coordinates, shape (nb_atoms,3)
in	<i>nb_atoms</i>	Number of atoms / points

9.1.1.2 calculate_distance()

```
subroutine maths::calculate_distance (
    real (kind=8), dimension(3), intent(in) p1,
    real (kind=8), dimension(3), intent(in) p2,
    real (kind=8), intent(out) dist )
```

Compute Euclidean distance between two 3D points.

Parameters

in	<i>p1</i>	First point (3 elements)
in	<i>p2</i>	Second point (3 elements)
out	<i>dist</i>	Euclidean distance $\ p1 - p2\ $

9.1.1.3 cross()

```
subroutine maths::cross (
    real ( kind=8 ), dimension ( 3 ), intent(in) v1,
    real ( kind=8 ), dimension ( 3 ), intent(in) v2,
    real ( kind=8 ), dimension ( 3 ), intent(out) v3 )
```

Compute the cross product of two 3D vectors.

Computes $v3 = v1 \times v2$.

Parameters

in	<i>v1</i>	Left-hand operand vector (3 elements)
in	<i>v2</i>	Right-hand operand vector (3 elements)
out	<i>v3</i>	Resulting cross product (3 elements)

9.1.1.4 update_complex()

```
subroutine maths::update_complex (
    real ( kind = 8 ), dimension ( 3 ), intent(in) xc1,
    real ( kind = 8 ), dimension ( 3 ), intent(in) xc2,
    real ( kind = 8 ), dimension ( 3 ), intent(in) trans,
    real ( kind = 8 ), dimension ( 3 ), intent(in) rot_vx,
    real ( kind = 8 ), dimension ( 3 ), intent(in) rot_vy,
    integer, intent(in) nb_atoms,
    real ( kind = 8 ), dimension (:, :), intent(in) coords,
    real ( kind = 8 ), dimension (:, :), intent(out) new_coords )
```

Apply translation and rotation to a set of atomic coordinates.

Transforms the coordinates of a complex. The routine first recenters coordinates to the origin with *xc2*, applies a rotation defined by *rot_vx* and *rot_vy* (their cross product defines the third axis), then applies an additional translation *trans*, moves to the new frame centered at *xc1*, and writes the results to *new_coords*.

Parameters

in	<i>xc1</i>	Target center position (3)
in	<i>xc2</i>	Original center position (3)
in	<i>trans</i>	Translation to apply after rotation (3)
in	<i>rot_vx</i>	First rotation axis vector (3)
in	<i>rot_vy</i>	Second rotation axis vector (3)
in	<i>nb_atoms</i>	Number of atoms (integer)
in	<i>coords</i>	Input coordinates, shape (nb_atoms,3)
out	<i>new_coords</i>	Output coordinates, shape (nb_atoms,3)

9.1.1.5 `vectors_angle_2d()`

```
subroutine maths::vectors_angle_2d (
    real( kind=8 ), dimension( 3 ), intent(in) vector1,
    real( kind=8 ), dimension( 3 ), intent(in) vector2,
    real (kind=8), intent(out) theta_degrees )
```

Compute the angle in degrees between two vectors projected to 2D.

Uses only the first two components of the input vectors to compute the angle in the XY plane. Returns 0 if either vector has zero length in the XY plane.

Parameters

in	<i>vector1</i>	First vector (use components 1 and 2)
in	<i>vector2</i>	Second vector (use components 1 and 2)
out	<i>theta_degrees</i>	Angle between projected vectors in degrees

9.1.1.6 `vectors_angle_3d()`

```
subroutine maths::vectors_angle_3d (
    real( kind=8 ), dimension( 3 ), intent(in) vector1,
    real( kind=8 ), dimension( 3 ), intent(in) vector2,
    real (kind=8), intent(out) theta_degrees )
```

Compute the angle in degrees between two 3D vectors.

Calculates the angle (in degrees) between `vector1` and `vector2` using the dot product formula and `acos`.

Parameters

in	<i>vector1</i>	First 3D vector (3 elements)
in	<i>vector2</i>	Second 3D vector (3 elements)
out	<i>theta_degrees</i>	Angle between vectors in degrees

9.2 mod_array Module Reference

Functions/Subroutines

- subroutine [array_z_coord](#) (complexes, array_sorted, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, solute←_crds)
Compute the Z-coordinate (along transformed z-axis) for each encounter and sort.
- subroutine [array_atoms_dist](#) (complexes, array_sorted, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, solute1_crds, solute2_crds)
Compute minimum inter-atomic distance between two solutes for each encounter and sort.
- subroutine [array_angle](#) (complexes, array_sorted, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, point1a, point1b, point2a, point2b, dimensions)
Compute angle between two vectors (defined by atom groups) per encounter and sort.
- subroutine [write_array](#) (array, filename)
Write a numeric array to a formatted text file.
- subroutine [read_array](#) (array, n, filename)
Read a numeric array from a formatted text file.
- subroutine [merge_sorted_segments](#) (arr, complexes, tmp_arr, tmp_comp, left, mid, right)
Merge two adjacent sorted segments of arrays in place.

9.2.1 Function/Subroutine Documentation

9.2.1.1 array_angle()

```
subroutine mod_array::array_angle (
    type(type_assoc_file), intent(inout) complexes,
    real(kind=8), dimension(n), intent(out) array_sorted,
    integer, intent(in) n,
    integer, intent(in) nb_atoms,
    real(kind=8), dimension(3), intent(in) xc1,
    real(kind=8), dimension(3), intent(in) xc2,
    real(kind=8), dimension(:, :), intent(in) trans_vector,
    real(kind=8), dimension(:, :), intent(in) rot1,
    real(kind=8), dimension(:, :), intent(in) rot2,
    real(kind=8), dimension(:), intent(in) point1a,
    real(kind=8), dimension(:), intent(in) point1b,
    real(kind=8), dimension(:), intent(in) point2a,
    real(kind=8), dimension(:), intent(in) point2b,
    integer, intent(in) dimensions )
```

Compute angle between two vectors (defined by atom groups) per encounter and sort.

Computes the angle between two vectors (defined by atom groups) for each encounter. The resulting array is sorted from minimum to maximum value.

Parameters

in, out	<i>complexes</i>	Association object reordered consistently with sorted outputs
out	<i>array_sorted</i>	Output array of length <i>n</i> with angles (sorted)
in	<i>n</i>	Number of encounters
in	<i>nb_atoms</i>	Number of atoms used for transformation
in	<i>xc1,xc2</i>	Solute centers used for translation transformations
in	<i>trans_vector</i>	Translation vectors for each encounter (<i>n</i> , 3)

Parameters

in	<i>rot1,rot2</i>	Rotation vectors for each encounter (n, 3)
in	<i>point1a</i>	First point defining reference vector
in	<i>point1b</i>	Second point defining reference vector
in	<i>point2a</i>	First point of vector to be transformed
in	<i>point2b</i>	Second point of vector to be transformed
in	<i>dimensions</i>	Use 2 or 3 to select projection

9.2.1.2 array_atoms_dist()

```

subroutine mod_array::array_atoms_dist (
    type(type_assoc_file), intent(inout) complexes,
    real(kind=8), dimension(n), intent(out) array_sorted,
    integer, intent(in) n,
    integer, intent(in) nb_atoms,
    real(kind=8), dimension(3), intent(in) xc1,
    real(kind=8), dimension(3), intent(in) xc2,
    real(kind=8), dimension(:, :), intent(in) trans_vector,
    real(kind=8), dimension(:, :), intent(in) rot1,
    real(kind=8), dimension(:, :), intent(in) rot2,
    real(kind=8), dimension(:, :), intent(in) solute1_crds,
    real(kind=8), dimension(:, :), intent(in) solute2_crds )

```

Compute minimum inter-atomic distance between two solutes for each encounter and sort.

For each encounter, the second solute is transformed and the minimum distance to *solute1_crds* is computed and stored in the output array. The resulting array is sorted from minimum to maximum value.

Parameters

in, out	<i>complexes</i>	Association object reordered consistently with sorted outputs
out	<i>array_sorted</i>	Output array of length n with minimum distances (sorted)
in	<i>n</i>	Number of encounters
in	<i>nb_atoms</i>	Number of atoms in <i>solute2_crds</i>
in	<i>xc1,xc2</i>	Solute centers used for translation transformations
in	<i>trans_vector</i>	Translation vectors for each encounter (n, 3)
in	<i>rot1,rot2</i>	Rotation vectors for each encounter (n, 3)
in	<i>solute1_crds</i>	Coordinates of solute1 atoms
in	<i>solute2_crds</i>	Coordinates of solute2 atoms

9.2.1.3 array_z_coord()

```

subroutine mod_array::array_z_coord (
    type(type_assoc_file), intent(inout) complexes,
    real(kind=8), dimension(n), intent(out) array_sorted,
    integer, intent(in) n,
    integer, intent(in) nb_atoms,
    real(kind=8), dimension(3), intent(in) xc1,

```

```

real(kind=8), dimension(3), intent(in) xc2,
real(kind=8), dimension(:, :), intent(in) trans_vector,
real(kind=8), dimension(:, :), intent(in) rot1,
real(kind=8), dimension(:, :), intent(in) rot2,
real(kind=8), dimension(:, :), intent(in) solute_crds )

```

Compute the Z-coordinate (along transformed z-axis) for each encounter and sort.

Computes the Z coordinate for each encounter and fills the output array. The resulting array is sorted from minimum to maximum value.

Parameters

in, out	<i>complexes</i>	Association object reordered consistently with sorted outputs
out	<i>array_sorted</i>	Output array of length <i>n</i> with z-coordinates (sorted)
in	<i>n</i>	Number of encounters
in	<i>nb_atoms</i>	Number of atoms in <i>solute_crds</i>
in	<i>xc1,xc2</i>	Solute centers used for translation transformations
in	<i>trans_vector</i>	Translation vectors for each encounter (<i>n</i> , 3)
in	<i>rot1,rot2</i>	Rotation vectors for each encounter (<i>n</i> , 3)
in	<i>solute_crds</i>	Coordinates of solute atoms (<i>nb_atoms</i> , 3)

9.2.1.4 merge_sorted_segments()

```

subroutine mod_array::merge_sorted_segments (
    real(kind=8), dimension(:), intent(inout) arr,
    type(type_assoc_file), intent(inout) complexes,
    real(kind=8), dimension(:), intent(inout) tmp_arr,
    character(len=217), dimension(:), intent(inout) tmp_comp,
    integer, intent(in) left,
    integer, intent(in) mid,
    integer, intent(in) right )

```

Merge two adjacent sorted segments of arrays in place.

Merges `arr(left:mid)` and `arr(mid+1:right)` into a single sorted segment `arr(left:right)`, re-ordering complexes and transform fields in the same way.

Parameters

in, out	<i>arr</i>	Array of values being sorted
in, out	<i>complexes</i>	Association object reordered alongside <i>arr</i>
out	<i>tmp_arr</i>	Temporary buffer for values (at least size <i>right</i>)
out	<i>tmp_comp</i>	Temporary buffer for companion (at least size <i>right</i>)
in	<i>left</i>	Start index of the first segment
in	<i>mid</i>	End index of the first segment
in	<i>right</i>	End index of the second segment

9.2.1.5 read_array()

```
subroutine mod_array::read_array (
    real(kind=8), dimension(:), intent(out), allocatable array,
    integer, intent(inout) n,
    character*128, intent(in) filename )
```

Read a numeric array from a formatted text file.

Parses a single-row formatted array file produced by write_array.

Parameters

out	array	Output array read from file (allocated)
in, out	n	Number of encounters (adjusted if file is smaller)
in	filename	Input filename

9.2.1.6 write_array()

```
subroutine mod_array::write_array (
    real(kind=8), dimension(:), intent(in) array,
    character*128, intent(in) filename )
```

Write a numeric array to a formatted text file.

Writes array as a single formatted line using F10.4 floats.

Parameters

in	array	Input array to write
in	filename	Output filename

9.3 mod_assoc Module Reference

Data Types

- type [type_assoc_file](#)
Data type for storing binned data (eg.

Functions/Subroutines

- subroutine [allocate_assoc_object](#) (this, nlines)
Allocate storage for an association/complexes object.
- integer function [size_assoc](#) (assoc_file)
Count encounter records in an association/complexes file.
- subroutine [fill_assoc_object](#) (this, assoc_file)
Read an association/complexes file and populate [type_assoc_file](#).
- subroutine [write_complexes](#) (complexes, nb_encounters, complexes_filename)

9.3.1 Function/Subroutine Documentation

9.3.1.1 `allocate_assoc_object()`

```
subroutine mod_assoc::allocate_assoc_object (
    type(type_assoc_file), intent(inout) this,
    integer, intent(in) nlines )
```

Allocate storage for an association/complexes object.

Allocates the arrays inside `this` to hold `nlines` encounter records and initializes numeric arrays to zero.

Parameters

in, out	<i>this</i>	Instance of <code>type_assoc_file</code> to initialize
in	<i>nlines</i>	Number of encounter lines to allocate

9.3.1.2 `fill_assoc_object()`

```
subroutine mod_assoc::fill_assoc_object (
    type(type_assoc_file), intent(inout) this,
    integer, intent(in) assoc_file )
```

Read an association/complexes file and populate `type_assoc_file`.

Reads header lines from the open unit `assoc_file` and extracts the two reference centers (`xc1`, `xc2`) from header lines 3 and 4. Subsequent non-header lines are stored into `thislines` and the translation and rotation vectors are parsed into `thistrans_vector`, `thisrot1` and `thisrot2`.

Parameters

in, out	<i>this</i>	Allocated <code>type_assoc_file</code> to populate
in	<i>assoc_file</i>	Fortran unit number of an already-open file

9.3.1.3 `size_assoc()`

```
integer function mod_assoc::size_assoc (
    integer assoc_file )
```

Count encounter records in an association/complexes file.

Scans `assoc_file` (a Fortran unit number) and counts the number of non-comment lines. Lines starting with `#` are treated as comments and ignored. Returns the number of encounter records.

Parameters

in	<i>assoc_file</i>	Fortran unit number of an already-open file
----	-------------------	---

Returns

nlines Number of encounter records found

9.3.1.4 write_complexes()

```
subroutine mod_assoc::write_complexes (
    type(type_assoc_file), intent(in) complexes,
    integer, intent(in) nb_encounters,
    character(len=*), intent(in) complexes_filename )
```

9.4 mod_clust_algorithm Module Reference**Functions/Subroutines**

- subroutine [linkage_clustering](#) (array, n, output_name, complexes)
Perform hierarchical clustering on a distance array.
- subroutine [write_cluster_elements](#) (tot_encounters, cluster_parent, cluster_count, active_clusters, output_name)
Write cluster membership lists to a text file.
- subroutine [write_clust_info](#) (tot_encounters, representative_indexes, active_clusters, clust_count, cluster_average, cluster_sd, output_name)
Write summary information about clusters to a text file.
- subroutine [write_cluster_complexes](#) (tot_encounters, cluster_parent, cluster_count, active_clusters, output_name, complexes)
Write complexes contained in each cluster to individual files.

9.4.1 Function/Subroutine Documentation**9.4.1.1 linkage_clustering()**

```
subroutine mod_clust_algorithm::linkage_clustering (
    real (kind=8), dimension(:), intent(inout), allocatable array,
    integer, intent(in) n,
    character*128, intent(in) output_name,
    type(type_assoc_file) complexes )
```

Perform hierarchical clustering on a distance array.

This routine performs hierarchical clustering using mean linkage on the input `array` (shape `(n)`) of sorted per-encounter values. The routine mutates `array` during merging and terminates merging when clusters are farther apart than an internally computed threshold. Results are written to files named using `output_name` and complexes information is optionally recorded via the `complexes` structure.

Parameters

in, out	<code>array</code>	Array of per-encounter values (n) This array is updated during clustering
in	<code>n</code>	Number of elements / encounters
in	<code>output_name</code>	Base name for output files
in	<code>complexes</code>	Structure containing complex text lines/headers

9.4.1.2 write_clust_info()

```
subroutine mod_clust_algorithm::write_clust_info (
    integer, intent(in) tot_encounters,
    integer, dimension(:), intent(in) representative_indexes,
    logical, dimension(:), intent(in) active_clusters,
    integer, dimension(tot_encounters), intent(in) clust_count,
    real (kind=8), dimension(tot_encounters), intent(in) cluster_average,
    real (kind=8), dimension(tot_encounters), intent(in) cluster_sd,
    character*128, intent(in) output_name )
```

Write summary information about clusters to a text file.

Produces a summary file <output_name>_clust_info.txt listing cluster population, representative index, average and standard deviation for each active cluster.

Parameters

in	<i>tot_encounters</i>	Total number of encounters
in	<i>representative_indexes</i>	Representative index per cluster
in	<i>active_clusters</i>	Logical mask indicating active clusters
in	<i>clust_count</i>	Array with population of each cluster
in	<i>cluster_average</i>	Per-cluster average values
in	<i>cluster_sd</i>	Per-cluster standard deviations
in	<i>output_name</i>	Base filename for outputs

9.4.1.3 write_cluster_complexes()

```
subroutine mod_clust_algorithm::write_cluster_complexes (
    integer, intent(in) tot_encounters,
    integer, dimension(:), intent(in) cluster_parent,
    integer, dimension(:), intent(in) cluster_count,
    logical, dimension(:), intent(in) active_clusters,
    character*128, intent(in) output_name,
    type(type_assoc_file) complexes )
```

Write complexes contained in each cluster to individual files.

For each active cluster this routine creates a file <output_name>_clusterXXXX_complexes containing the header lines from complexeshead and the encounter lines for the cluster members taken from complexeslines.

Parameters

in	<i>tot_encounters</i>	Number of clusters/encounters
in	<i>cluster_parent</i>	Parent cluster for each encounter
in	<i>cluster_count</i>	Number of members per cluster
in	<i>active_clusters</i>	Logical mask indicating active clusters
in	<i>output_name</i>	Base filename for output files
in	<i>complexes</i>	Structure with header and lines to write

9.4.1.4 write_cluster_elements()

```
subroutine mod_clust_algorithm::write_cluster_elements (
    integer, intent(in) tot_encounters,
    integer, dimension(:), intent(in) cluster_parent,
    integer, dimension(:), intent(in) cluster_count,
    logical, dimension(:), intent(in) active_clusters,
    character*128, intent(in) output_name )
```

Write cluster membership lists to a text file.

Outputs a file named <output_name>_clusters.txt containing the members of each active cluster in a tabular text format.

Parameters

in	<i>tot_encounters</i>	Number of possible clusters/encounters
in	<i>cluster_parent</i>	Parent cluster for each encounter
in	<i>cluster_count</i>	Number of members per cluster
in	<i>active_clusters</i>	Logical mask indicating active clusters
in	<i>output_name</i>	Base filename for output

9.5 mod_pdb Module Reference

Data Types

- type [type_pdb_atom](#)
- type [type_pdb_chain](#)
- type [type_pdb_file](#)

Data type for storing binned data (eg.

- type [type_pdb_residue](#)

Functions/Subroutines

- subroutine [allocate_pdb_object](#) (this, pdb_file, natoms, nresidues, nchains)
Allocate and initialize a [type_pdb_file](#) structure from a PDB file.
- integer function [count_atoms](#) (pdb_file)
Count ATOM/HETATM records in a PDB file.
- integer function [count_residues](#) (pdb_file)
Count residues in a PDB file.
- integer function [count_chains](#) (pdb_file)
Count chain separators (TER records) in a PDB file.
- subroutine [fill_pdb_object](#) (this, pdb_file)
Parse PDB records and populate the [type_pdb_file](#) structure.

9.5.1 Function/Subroutine Documentation

9.5.1.1 allocate_pdb_object()

```
subroutine mod_pdb::allocate_pdb_object (
    type(type_pdb_file), intent(inout) this,
    integer, intent(in)   pdb_file,
    integer, intent(in)   natoms,
    integer, intent(in)   nresidues,
    integer, intent(in)   nchains )
```

Allocate and initialize a `type_pdb_file` structure from a PDB file.

This routine allocates the storage for `thisatoms`, `thisresidues` and `thischains` using the supplied counts and performs a preliminary scan of the open Fortran unit `pdb_file` to determine per-chain and per-residue allocation sizes for nested arrays.

Parameters

in, out	<i>this</i>	<code>type_pdb_file</code> instance to allocate and initialize
in	<i>pdb_file</i>	Fortran unit number of an already-open PDB file
in	<i>natoms</i>	Total number of ATOM/HETATM records expected
in	<i>nresidues</i>	Number of residues expected
in	<i>nchains</i>	Number of chains expected

9.5.1.2 count_atoms()

```
integer function mod_pdb::count_atoms (
    integer pdb_file )
```

Count ATOM/HETATM records in a PDB file.

Scans the open Fortran file unit `pdb_file` and returns the number of lines starting with ATOM or HETATM.

Parameters

in	<i>pdb_file</i>	Fortran unit number of an already-open PDB file
----	-----------------	---

Returns

`natoms` Number of ATOM/HETATM records found

9.5.1.3 count_chains()

```
integer function mod_pdb::count_chains (
    integer pdb_file )
```

Count chain separators (TER records) in a PDB file.

Scans the open Fortran file unit `pdb_file` and counts the number of TER records, which are treated as chain separators. Returns the number of chains found.

Parameters

in	<i>pdb_file</i>	Fortran unit number of an already-open PDB file
----	-----------------	---

Returns

nchains Number of chains (TER records) found

9.5.1.4 count_residues()

```
integer function mod_pdb::count_residues (
    integer pdb_file )
```

Count residues in a PDB file.

Scans the open Fortran file unit *pdb_file* and counts residue changes by comparing residue identifiers in ATOM/HETATM records. Returns the number of distinct residues found.

Parameters

in	<i>pdb_file</i>	Fortran unit number of an already-open PDB file
----	-----------------	---

Returns

nresidues Number of residues found

9.5.1.5 fill_pdb_object()

```
subroutine mod_pdb::fill_pdb_object (
    type(type_pdb_file), intent(inout) this,
    integer, intent(in) pdb_file )
```

Parse PDB records and populate the [type_pdb_file](#) structure.

Reads ATOM/HETATM records from the already-open Fortran unit *pdb_file*, fills *this* atoms with atom records and populates *this*residues and *this*chains so that residues contain their per-residue atoms arrays and chains contain their residues and atoms. The routine expects *this* to have been allocated with the correct sizes (see [allocate_pdb_object](#)).

Parameters

in, out	<i>this</i>	Pre-allocated type_pdb_file to populate
in	<i>pdb_file</i>	Fortran unit number of an already-open PDB file

9.6 mod_threshold Module Reference

Functions/Subroutines

- subroutine `array_z_coord` (array, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, solute_crds)
Compute the Z-coordinate (along transformed z-axis) for each encounter.
- subroutine `array_atoms_dist` (array, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, solute1_crds, solute2_crds)
Compute minimum inter-atomic distance between two solutes for each encounter.
- subroutine `array_angle` (array, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, point1a, point1b, point2a, point2b, dimensions)
Compute angle between two vectors (defined by atom groups) per encounter.
- subroutine `sort_array` (arr, encounter_indexes)
Sort an array in ascending order and reorder encounter indexes accordingly.

9.6.1 Function/Subroutine Documentation

9.6.1.1 array_angle()

```
subroutine mod_threshold::array_angle (
    real(kind=8), dimension(n), intent(out) array,
    integer, intent(in) n,
    integer, intent(in) nb_atoms,
    real(kind=8), dimension(3), intent(in) xc1,
    real(kind=8), dimension(3), intent(in) xc2,
    real(kind=8), dimension(:, :), intent(in) trans_vector,
    real(kind=8), dimension(:, :), intent(in) rot1,
    real(kind=8), dimension(:, :), intent(in) rot2,
    real(kind=8), dimension(:), intent(in) point1a,
    real(kind=8), dimension(:), intent(in) point1b,
    real(kind=8), dimension(:), intent(in) point2a,
    real(kind=8), dimension(:), intent(in) point2b,
    integer, intent(in) dimensions )
```

Compute angle between two vectors (defined by atom groups) per encounter.

Transforms the pair of points from `solute2` for each encounter and computes the angle between the vector defined by `point1b`-`point1a` and the vector defined by the transformed `point2b`-`point2a`.

Parameters

out	<code>array</code>	Output array of length <code>n</code> with angles in degrees
in	<code>n</code>	Number of encounters
in	<code>nb_atoms</code>	Number of atoms used for transformation
in	<code>xc1,xc2</code>	Centers used for transform
in	<code>trans_vector</code>	Translation vectors for each encounter (<code>n</code> , 3)
in	<code>rot1,rot2</code>	Rotation vectors for each encounter (<code>n</code> , 3)
in	<code>point1a</code>	First point defining reference vector
in	<code>point1b</code>	Second point defining reference vector
in	<code>point2a</code>	First point of vector to be transformed
in	<code>point2b</code>	Second point of vector to be transformed
in	<code>dimensions</code>	Use 2 or 3 to select projection

9.6.1.2 array_atoms_dist()

```

subroutine mod_threshold::array_atoms_dist (
    real(kind=8), dimension(n), intent(out) array,
    integer, intent(in) n,
    integer, intent(in) nb_atoms,
    real(kind=8), dimension(3), intent(in) xc1,
    real(kind=8), dimension(3), intent(in) xc2,
    real(kind=8), dimension(:, :), intent(in) trans_vector,
    real(kind=8), dimension(:, :), intent(in) rot1,
    real(kind=8), dimension(:, :), intent(in) rot2,
    real(kind=8), dimension(:, :), intent(in) solute1_crds,
    real(kind=8), dimension(:, :), intent(in) solute2_crds )

```

Compute minimum inter-atomic distance between two solutes for each encounter.

For each encounter, the second solute is transformed and the minimum distance between any atom of `solute1_crds` and the transformed `solute2_crds` is computed and stored in `array`.

Parameters

out	<i>array</i>	Output array of length <i>n</i> with minimum distances
in	<i>n</i>	Number of encounters
in	<i>nb_atoms</i>	Number of atoms in <code>solute2_crds</code>
in	<i>xc1,xc2</i>	Centers used for transform
in	<i>trans_vector</i>	Translation vectors for each encounter (<i>n</i> , 3)
in	<i>rot1,rot2</i>	Rotation vectors for each encounter (<i>n</i> , 3)
in	<i>solute1_crds</i>	Coordinates of solute1 atoms
in	<i>solute2_crds</i>	Coordinates of solute2 atoms

9.6.1.3 array_z_coord()

```

subroutine mod_threshold::array_z_coord (
    real(kind=8), dimension(n), intent(out) array,
    integer, intent(in) n,
    integer, intent(in) nb_atoms,
    real(kind=8), dimension(3), intent(in) xc1,
    real(kind=8), dimension(3), intent(in) xc2,
    real(kind=8), dimension(:, :), intent(in) trans_vector,
    real(kind=8), dimension(:, :), intent(in) rot1,
    real(kind=8), dimension(:, :), intent(in) rot2,
    real(kind=8), dimension(:, :), intent(in) solute_crds )

```

Compute the Z-coordinate (along transformed z-axis) for each encounter.

Recenters each encounter at `xc2`, applies rotation/translation using `rot1/rot2` and `trans_vector`, and extracts the Z coordinate of the first atom in `solute_crds` for every encounter into `array`.

Parameters

out	<i>array</i>	Output array of length <i>n</i> with z-coordinates
in	<i>n</i>	Number of encounters
in	<i>nb_atoms</i>	Number of atoms in <code>solute_crds</code>

Parameters

in	<i>xc1,xc2</i>	Centers used for transform
in	<i>trans_vector</i>	Translation vectors for each encounter (n, 3)
in	<i>rot1,rot2</i>	Rotation vectors for each encounter (n, 3)
in	<i>solute_crds</i>	Coordinates of solute atoms (nb_atoms, 3)

9.6.1.4 sort_array()

```
subroutine mod_threshold::sort_array (
    real (kind=8), dimension(:), intent(inout) arr,
    integer, dimension(:), intent(inout) encounter_indexes )
```

Sort an array in ascending order and reorder encounter indexes accordingly.

Simple selection sort used to reorder `arr` and the parallel `encounter_indexes` array. Intended for small to moderate sizes.

Parameters

in, out	<i>arr</i>	Array to sort (modified in-place)
in, out	<i>encounter_indexes</i>	Parallel index array reordered to match arr

9.7 read_input Module Reference

Functions/Subroutines

- subroutine [read_pdb](#) (pdb, filename, tot_atoms, tot_residues, tot_chains)
Read a PDB file into a `type_pdb_file` object.
- subroutine [read_assoc](#) (assoc, filename, tot_encounters)
Read an association/complexes file into a `type_assoc_file` object.

9.7.1 Function/Subroutine Documentation

9.7.1.1 read_assoc()

```
subroutine read_input::read_assoc (
    type ( type\_assoc\_file ) assoc,
    character*128, intent(in) filename,
    integer, intent(out) tot_encounters )
```

Read an association/complexes file into a `type_assoc_file` object.

Checks the file exists, opens it and uses `size_assoc`, `allocate_assoc_object` and `fill_assoc_↵` object to construct the `assoc` structure.

Parameters

out	<i>assoc</i>	Allocated and filled assoc object
in	<i>filename</i>	Path to complexes file
out	<i>tot_encounters</i>	Number of encounter records read

9.7.1.2 read_pdb()

```
subroutine read_input::read_pdb (  
    type ( type_pdb_file ) pdb,  
    character*128, intent(in) filename,  
    integer, intent(out) tot_atoms,  
    integer, intent(out) tot_residues,  
    integer, intent(out) tot_chains )
```

Read a PDB file into a `type_pdb_file` object.

Validates the existence of `filename`, opens it, determines the counts for atoms, residues and chains, allocates `pdb` and fills it using `allocate_pdb_object` and `fill_pdb_object`.

Parameters

out	<i>pdb</i>	Allocated and filled PDB object
in	<i>filename</i>	Path to PDB file
out	<i>tot_atoms</i>	Number of atoms found
out	<i>tot_residues</i>	Number of residues found
out	<i>tot_chains</i>	Number of chains found

Chapter 10

Data Type Documentation

10.1 `mod_assoc::type_assoc_file` Type Reference

Data type for storing binned data (eg.

Public Attributes

- integer `nlines`
Number of lines in assoc_file file.
- character(len=217), dimension(:), allocatable `lines`
- character(len=217), dimension(4) `head`
- real(kind=8), dimension(:, :), allocatable `trans_vector`
Translational vector.
- real(kind=8), dimension(:, :), allocatable `rot1`
rotX, rotY
- real(kind=8), dimension(:, :), allocatable `rot2`
- real(kind=8), dimension(3) `xc1`
center of mass of p2
- real(kind=8), dimension(3) `xc2`

10.1.1 Detailed Description

Data type for storing binned data (eg.

Radial distribution functions)

10.1.2 Member Data Documentation

10.1.2.1 `head`

```
character(len=217), dimension(4) mod_assoc::type_assoc_file::head
```

10.1.2.2 lines

```
character(len=217), dimension(:), allocatable mod_assoc::type_assoc_file::lines
```

10.1.2.3 nlines

```
integer mod_assoc::type_assoc_file::nlines
```

Number of lines in assoc_file file.

10.1.2.4 rot1

```
real (kind=8), dimension (:,:), allocatable mod_assoc::type_assoc_file::rot1
```

rotX, rotY

10.1.2.5 rot2

```
real (kind=8), dimension (:,:), allocatable mod_assoc::type_assoc_file::rot2
```

10.1.2.6 trans_vector

```
real (kind=8), dimension (:, :), allocatable mod_assoc::type_assoc_file::trans_vector
```

Translational vector.

10.1.2.7 xc1

```
real (kind=8), dimension (3) mod_assoc::type_assoc_file::xc1
```

center of mass of p2

10.1.2.8 xc2

```
real (kind=8), dimension (3) mod_assoc::type_assoc_file::xc2
```

The documentation for this type was generated from the following file:

- [mod_assoc.f90](#)

10.2 mod_pdb::type_pdb_atom Type Reference

Public Attributes

- integer `record`
Number of atoms in pdb file Flag for record type, 1 = ATOM, 2 = HETATM.
- real(kind=8), dimension(3) `coord`
Position, occupancy and temp factor - can be used to store other data eg in pqr.
- real(kind=8) `occ`
- real(kind=8) `beta`
- character(len=4) `name`
Atom name, res name and chain name.
- character(len=3) `resname`
- character(len=1) `chainid`
- integer `id`
Atom and residue numbers.
- integer `resid`

10.2.1 Member Data Documentation

10.2.1.1 beta

```
real (kind=8) mod_pdb::type_pdb_atom::beta
```

10.2.1.2 chainid

```
character (len=1) mod_pdb::type_pdb_atom::chainid
```

10.2.1.3 coord

```
real (kind=8), dimension (3) mod_pdb::type_pdb_atom::coord
```

Position, occupancy and temp factor - can be used to store other data eg in pqr.

10.2.1.4 id

```
integer mod_pdb::type_pdb_atom::id
```

Atom and residue numbers.

10.2.1.5 name

```
character (len=4) mod_pdb::type_pdb_atom::name
```

Atom name, res name and chain name.

10.2.1.6 occ

```
real (kind=8) mod_pdb::type_pdb_atom::occ
```

10.2.1.7 record

```
integer mod_pdb::type_pdb_atom::record
```

Number of atoms in pdb file Flag for record type, 1 = ATOM, 2 = HETATM.

10.2.1.8 resid

```
integer mod_pdb::type_pdb_atom::resid
```

10.2.1.9 resname

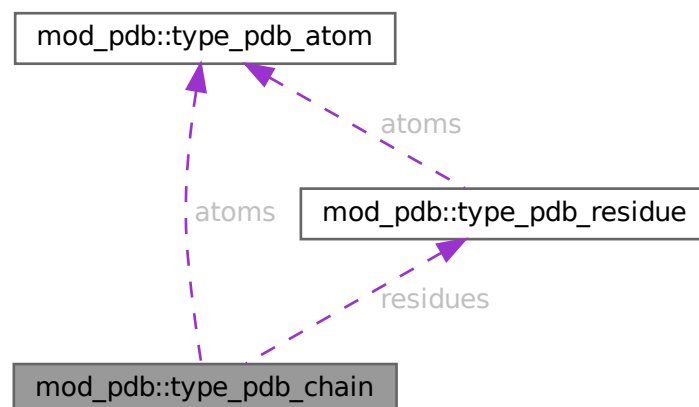
```
character (len=3) mod_pdb::type_pdb_atom::resname
```

The documentation for this type was generated from the following file:

- [mod_pdb.f90](#)

10.3 mod_pdb::type_pdb_chain Type Reference

Collaboration diagram for mod_pdb::type_pdb_chain:



Public Attributes

- integer [natoms](#)
Number of atoms in pdb file.
- integer [nresidues](#)
- character [chainid](#)
Chain id.
- type([type_pdb_atom](#)), dimension(:), allocatable [atoms](#)
Flag for record type, 1 = ATOM, 2 = HETATM.
- type([type_pdb_residue](#)), dimension(:), allocatable [residues](#)

10.3.1 Member Data Documentation

10.3.1.1 atoms

`type(type\_pdb\_atom), dimension (:), allocatable mod_pdb::type_pdb_chain::atoms`

Flag for record type, 1 = ATOM, 2 = HETATM.

10.3.1.2 chainid

`character mod_pdb::type_pdb_chain::chainid`

Chain id.

10.3.1.3 natoms

`integer mod_pdb::type_pdb_chain::natoms`

Number of atoms in pdb file.

10.3.1.4 nresidues

`integer mod_pdb::type_pdb_chain::nresidues`

10.3.1.5 residues

`type(type\_pdb\_residue), dimension (:), allocatable mod_pdb::type_pdb_chain::residues`

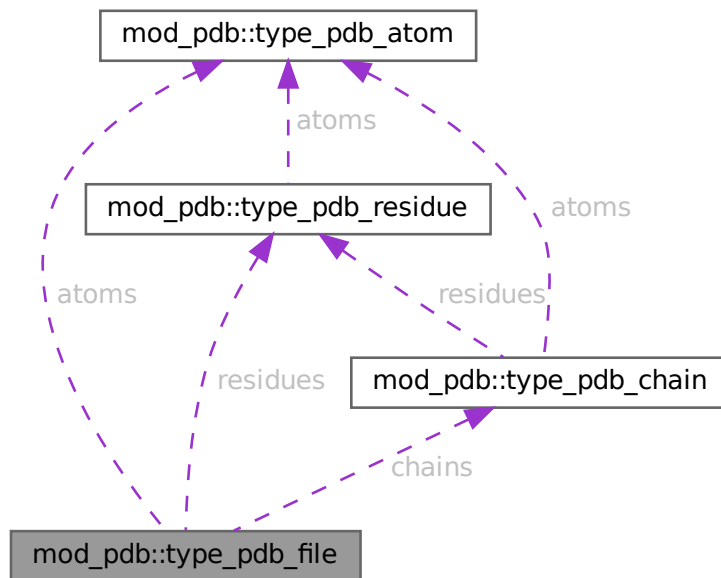
The documentation for this type was generated from the following file:

- [mod_pdb.f90](#)

10.4 mod_pdb::type_pdb_file Type Reference

Data type for storing binned data (eg.

Collaboration diagram for mod_pdb::type_pdb_file:



Public Attributes

- integer `natoms`
Number of atoms in pdb file.
- integer `nresidues`
- integer `nchains`
- type(`type_pdb_atom`), dimension(:), allocatable `atoms`
Flag for record type, 1 = ATOM, 2 = HETATM.
- type(`type_pdb_residue`), dimension(:), allocatable `residues`
- type(`type_pdb_chain`), dimension(:), allocatable `chains`

10.4.1 Detailed Description

Data type for storing binned data (eg.

Radial distribution functions)

10.4.2 Member Data Documentation

10.4.2.1 atoms

`type(type_pdb_atom)`, dimension (:), allocatable mod_pdb::type_pdb_file::atoms

Flag for record type, 1 = ATOM, 2 = HETATM.

10.4.2.2 chains

`type(type_pdb_chain)`, dimension (:), allocatable mod_pdb::type_pdb_file::chains

10.4.2.3 natoms

`integer mod_pdb::type_pdb_file::natoms`

Number of atoms in pdb file.

10.4.2.4 nchains

`integer mod_pdb::type_pdb_file::nchains`

10.4.2.5 nresidues

`integer mod_pdb::type_pdb_file::nresidues`

10.4.2.6 residues

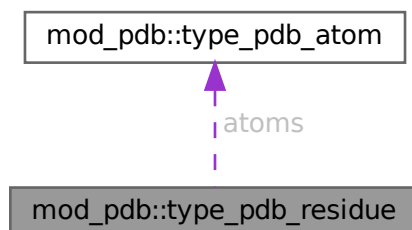
`type(type_pdb_residue)`, dimension (:), allocatable mod_pdb::type_pdb_file::residues

The documentation for this type was generated from the following file:

- [mod_pdb.f90](#)

10.5 mod_pdb::type_pdb_residue Type Reference

Collaboration diagram for mod_pdb::type_pdb_residue:



Public Attributes

- integer [natoms](#)
Number of atoms in pdb file.
- character [chainid](#)
Chain id.
- character(len=3) [resname](#)
Residue name.
- integer [resid](#)
Residue number.
- type([type_pdb_atom](#)), dimension(:), allocatable [atoms](#)
Flag for record type, 1 = ATOM, 2 = HETATM.

10.5.1 Member Data Documentation

10.5.1.1 atoms

```
type(type\_pdb\_atom), dimension (:), allocatable mod_pdb::type_pdb_residue::atoms
```

Flag for record type, 1 = ATOM, 2 = HETATM.

10.5.1.2 chainid

```
character mod_pdb::type_pdb_residue::chainid
```

Chain id.

10.5.1.3 natoms

```
integer mod_pdb::type_pdb_residue::natoms
```

Number of atoms in pdb file.

10.5.1.4 resid

```
integer mod_pdb::type_pdb_residue::resid
```

Residue number.

10.5.1.5 resname

```
character (len=3) mod_pdb::type_pdb_residue::resname
```

Residue name.

The documentation for this type was generated from the following file:

- [mod_pdb.f90](#)

Chapter 11

File Documentation

11.1 clust.f90 File Reference

Hierarchical clustering driver for encounter arrays.

Functions/Subroutines

- program [main](#)
- subroutine [parse_arguments](#) (p_complexes_filename, p_complexes_bool, p_nb_encounters, p_nb_↵
encounters_bool, p_datadist_filename, p_output_name, p_output_name_bool, p_help_bool)

Parse command-line arguments for `clust`.

11.1.1 Detailed Description

Hierarchical clustering driver for encounter arrays.

[clust.f90](#) - Hierarchical clustering for encounter arrays.

Usage: ./clust -help

Reads an array and runs hierarchical clustering with mean linkage. Results written to output_name files.

Author

Abraham Muñoz-Chicharro

Version

1.0

Date

2026-04-05

11.1.2 Function/Subroutine Documentation

11.1.2.1 main()

program main

11.1.2.2 parse_arguments()

```
subroutine main::parse_arguments (
    character*128, intent(out) p_complexes_filename,
    logical, intent(out) p_complexes_bool,
    integer, intent(out) p_nb_encounters,
    logical, intent(out) p_nb_encounters_bool,
    character*128, intent(out) p_datadist_filename,
    character*128, intent(out) p_output_name,
    logical, intent(out) p_output_name_bool,
    logical, intent(out) p_help_bool )
```

Parse command-line arguments for clust.

Parses all CLI flags and fills program configuration values.

Parameters

out	<i>p_complexes_filename</i>	Value parsed from <code>-complexes</code>
out	<i>p_complexes_bool</i>	Whether <code>-complexes</code> was provided
out	<i>p_nb_encounters</i>	Value parsed from <code>-nb_encounters</code>
out	<i>p_nb_encounters_bool</i>	Whether <code>-nb_encounters</code> was provided
out	<i>p_datadist_filename</i>	Value parsed from <code>-input</code>
out	<i>p_output_name</i>	Value parsed from <code>-output_name</code>
out	<i>p_output_name_bool</i>	Whether <code>-output_name</code> was provided
out	<i>p_help_bool</i>	Whether <code>-help</code> was provided

11.2 clust.md File Reference

11.3 contact_map.f90 File Reference

Residue proximity analysis across transformed encounters.

Functions/Subroutines

- program [main](#)
- subroutine [print_help](#) ()
Print usage/help text for `contact_map`.

11.3.1 Detailed Description

Residue proximity analysis across transformed encounters.

[contact_map.f90](#) - Find residues close to the surface across encounters.

Usage: ./contact_map -help

Reads a PDB and complexes file, transforms residue centers, and counts how many encounters bring residues within a threshold. Results: closest_residues.txt

Author

Abraham Muñoz-Chicharro

Version

1.0

Date

2026-04-05

11.3.2 Function/Subroutine Documentation

11.3.2.1 main()

```
program main
```

11.3.2.2 print_help()

```
subroutine main::print_help
```

Print usage/help text for `contact_map`.

Shows required command-line options and examples for running the residue analysis tool.

11.4 contact_map.md File Reference

11.5 make_data.f90 File Reference

Build encounter-based array data from structural inputs.

Functions/Subroutines

- program [main](#)
- subroutine [parse_arguments](#) (p_pdb1_filename, p_pdb1_bool, p_pdb2_filename, p_pdb2_bool, p_↵ complexes_filename, p_complexes_bool, p_nb_encounters, p_nb_encounters_bool, p_output_filename, p_output_bool, p_data_type, p_datatype_bool, p_arr_atoms1a, p_arr_atoms1a_bool, p_arr_atoms1b, p_↵ arr_atoms1b_bool, p_arr_atoms2a, p_arr_atoms2a_bool, p_arr_atoms2b, p_arr_atoms2b_bool, p_help_↵ bool)

Parse command-line arguments for `make_data`.

11.5.1 Detailed Description

Build encounter-based array data from structural inputs.

[make_data.f90](#) - Build encounter-based arrays from PDB and complex files.

Usage: `./make_data -help`

Reads PDB(s) and association/complex files, generates arrays (z_coord, atoms_dist, angle).

Author

Abraham Muñiz-Chicharro

Version

1.0

Date

2026-04-05

11.5.2 Function/Subroutine Documentation

11.5.2.1 `main()`

program main

11.5.2.2 parse_arguments()

```

subroutine main::parse_arguments (
    character*128, intent(out) p_pdb1_filename,
    logical, intent(out) p_pdb1_bool,
    character*128, intent(out) p_pdb2_filename,
    logical, intent(out) p_pdb2_bool,
    character*128, intent(out) p_complexes_filename,
    logical, intent(out) p_complexes_bool,
    integer, intent(out) p_nb_encounters,
    logical, intent(out) p_nb_encounters_bool,
    character*128, intent(out) p_output_filename,
    logical, intent(out) p_output_bool,
    character*128, intent(out) p_data_type,
    logical, intent(out) p_datatype_bool,
    character*128, dimension(:), intent(out), allocatable p_arr_atoms1a,
    logical, intent(out) p_arr_atoms1a_bool,
    character*128, dimension(:), intent(out), allocatable p_arr_atoms1b,
    logical, intent(out) p_arr_atoms1b_bool,
    character*128, dimension(:), intent(out), allocatable p_arr_atoms2a,
    logical, intent(out) p_arr_atoms2a_bool,
    character*128, dimension(:), intent(out), allocatable p_arr_atoms2b,
    logical, intent(out) p_arr_atoms2b_bool,
    logical, intent(out) p_help_bool )

```

Parse command-line arguments for make_data.

Parses all CLI flags and fills program configuration values. Atom-group flags consume multiple tokens until the next flag.

Parameters

out	<i>p_pdb1_filename</i>	Value parsed from -pdb1
out	<i>p_pdb1_bool</i>	Whether -pdb1 was provided
out	<i>p_pdb2_filename</i>	Value parsed from -pdb2
out	<i>p_pdb2_bool</i>	Whether -pdb2 was provided
out	<i>p_complexes_filename</i>	Value parsed from -complexes
out	<i>p_complexes_bool</i>	Whether -complexes was provided
out	<i>p_nb_encounters</i>	Value parsed from -nb_encounters
out	<i>p_nb_encounters_bool</i>	Whether -nb_encounters was provided
out	<i>p_output_filename</i>	Value parsed from -output
out	<i>p_output_bool</i>	Whether -output was provided
out	<i>p_data_type</i>	Value parsed from -data_type
out	<i>p_datatype_bool</i>	Whether -data_type was provided
out	<i>p_arr_atoms1a</i>	Atom list parsed from -atoms1 or -atoms1a
out	<i>p_arr_atoms1a_bool</i>	Whether -atoms1 or -atoms1a was provided
out	<i>p_arr_atoms1b</i>	Atom list parsed from -atoms1b
out	<i>p_arr_atoms1b_bool</i>	Whether -atoms1b was provided
out	<i>p_arr_atoms2a</i>	Atom list parsed from -atoms2 or -atoms2a
out	<i>p_arr_atoms2a_bool</i>	Whether -atoms2 or -atoms2a was provided
out	<i>p_arr_atoms2b</i>	Atom list parsed from -atoms2b
out	<i>p_arr_atoms2b_bool</i>	Whether -atoms2b was provided
out	<i>p_help_bool</i>	Whether -help was provided

11.6 make_data.md File Reference

11.7 maths.f90 File Reference

Mathematical and geometric utilities for clustering workflows.

Modules

- module [maths](#)

Functions/Subroutines

- subroutine [maths::cross](#) (v1, v2, v3)
Compute the cross product of two 3D vectors.
- subroutine [maths::update_complex](#) (xc1, xc2, trans, rot_vx, rot_vy, nb_atoms, coords, new_coords)
Apply translation and rotation to a set of atomic coordinates.
- subroutine [maths::vectors_angle_3d](#) (vector1, vector2, theta_degrees)
Compute the angle in degrees between two 3D vectors.
- subroutine [maths::vectors_angle_2d](#) (vector1, vector2, theta_degrees)
Compute the angle in degrees between two vectors projected to 2D.
- subroutine [maths::calculate_cog](#) (cog, coords, nb_atoms)
Compute the center of geometry (centroid) of a set of points.
- subroutine [maths::calculate_distance](#) (p1, p2, dist)
Compute Euclidean distance between two 3D points.

11.7.1 Detailed Description

Mathematical and geometric utilities for clustering workflows.

Mathematical utilities and coordinate transformation routines.

This module provides fundamental math operations including cross products, coordinate transformations, angle calculations, and rotation matrices.

Author

Abraham Muñoz-Chicharro

Version

1.0

Date

2026-04-05

11.8 mod_array.f90 File Reference

Build/read/write encounter-based arrays and per-encounter metrics.

Modules

- module [mod_array](#)

Functions/Subroutines

- subroutine [mod_array::array_z_coord](#) (complexes, array_sorted, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, solute_crds)
Compute the Z-coordinate (along transformed z-axis) for each encounter and sort.
- subroutine [mod_array::array_atoms_dist](#) (complexes, array_sorted, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, solute1_crds, solute2_crds)
Compute minimum inter-atomic distance between two solutes for each encounter and sort.
- subroutine [mod_array::array_angle](#) (complexes, array_sorted, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, point1a, point1b, point2a, point2b, dimensions)
Compute angle between two vectors (defined by atom groups) per encounter and sort.
- subroutine [mod_array::write_array](#) (array, filename)
Write a numeric array to a formatted text file.
- subroutine [mod_array::read_array](#) (array, n, filename)
Read a numeric array from a formatted text file.
- subroutine [mod_array::merge_sorted_segments](#) (arr, complexes, tmp_arr, tmp_comp, left, mid, right)
Merge two adjacent sorted segments of arrays in place.

11.8.1 Detailed Description

Build/read/write encounter-based arrays and per-encounter metrics.

Provides routines to compute z-coordinate, distance, and angle-based arrays from encounter transformations, as well as array I/O and sorting helpers.

Author

Abraham Muñoz-Chicharro

Version

1.0

Date

2026-04-05

11.9 mod_assoc.f90 File Reference

Association/complexes file data structures and parsing routines.

Data Types

- type `mod_assoc::type_assoc_file`
Data type for storing binned data (eg.

Modules

- module `mod_assoc`

Functions/Subroutines

- subroutine `mod_assoc::allocate_assoc_object` (this, nlines)
Allocate storage for an association/complexes object.
- integer function `mod_assoc::size_assoc` (assoc_file)
Count encounter records in an association/complexes file.
- subroutine `mod_assoc::fill_assoc_object` (this, assoc_file)
Read an association/complexes file and populate `type_assoc_file`.
- subroutine `mod_assoc::write_complexes` (complexes, nb_encounters, complexes_filename)

11.9.1 Detailed Description

Association/complexes file data structures and parsing routines.

Association/complexes file utilities.

This module defines the `type_assoc_file` derived type and routines to allocate and populate it from an association/complexes text file. The `type_assoc_file` stores the raw file lines, header lines, per-encounter translation vectors and rotation vectors, and the two reference centers `xc1` and `xc2` extracted from the file header.

NOTE: The routines in this file are adapted from the SDA7 project's `mod_pdb.f90` (original author Neil Bruce). This file contains code and ideas derived from that implementation. Appropriate attribution to the SDA7 sources is retained here. Consult the SDA7 project for original authorship and licensing details.

Author

Abraham Muñiz-Chicharro

Version

1.0

Date

2026-04-05

11.10 mod_clust_algorithm.f90 File Reference

Hierarchical clustering algorithms and output helpers.

Modules

- module [mod_clust_algorithm](#)

Functions/Subroutines

- subroutine [mod_clust_algorithm::linkage_clustering](#) (array, n, output_name, complexes)
Perform hierarchical clustering on a distance array.
- subroutine [mod_clust_algorithm::write_cluster_elements](#) (tot_encounters, cluster_parent, cluster_count, active_clusters, output_name)
Write cluster membership lists to a text file.
- subroutine [mod_clust_algorithm::write_clust_info](#) (tot_encounters, representative_indexes, active_clusters, clust_count, cluster_average, cluster_sd, output_name)
Write summary information about clusters to a text file.
- subroutine [mod_clust_algorithm::write_cluster_complexes](#) (tot_encounters, cluster_parent, cluster_count, active_clusters, output_name, complexes)
Write complexes contained in each cluster to individual files.

11.10.1 Detailed Description

Hierarchical clustering algorithms and output helpers.

11.11 mod_pdb.f90 File Reference

PDB data structures and parsing/allocation routines.

Data Types

- type [mod_pdb::type_pdb_file](#)
Data type for storing binned data (eg.
- type [mod_pdb::type_pdb_chain](#)
- type [mod_pdb::type_pdb_residue](#)
- type [mod_pdb::type_pdb_atom](#)

Modules

- module [mod_pdb](#)

Functions/Subroutines

- subroutine [mod_pdb::allocate_pdb_object](#) (this, pdb_file, natoms, nresidues, nchains)
Allocate and initialize a [type_pdb_file](#) structure from a PDB file.
- integer function [mod_pdb::count_atoms](#) (pdb_file)
Count ATOM/HETATM records in a PDB file.
- integer function [mod_pdb::count_residues](#) (pdb_file)
Count residues in a PDB file.
- integer function [mod_pdb::count_chains](#) (pdb_file)
Count chain separators (TER records) in a PDB file.
- subroutine [mod_pdb::fill_pdb_object](#) (this, pdb_file)
Parse PDB records and populate the [type_pdb_file](#) structure.

11.11.1 Detailed Description

PDB data structures and parsing/allocation routines.

Module to define a pdb type and pdb reading routine for tools.

NOTE: This implementation is based on the SDA7 project's [mod_pdb.f90](#) (original author Neil Bruce). The present file has been modified and expanded for the clustering tools in this workspace. Notable enhancements include the addition of `count_residues` and `count_chains` functions and changes that allow creation of richer in-memory objects where residues contain their atoms and chains contain residues and atoms. These changes make it possible to build `type_pdb_file` instances with proper `residues` and `chains` substructures (each storing their atoms) rather than storing atoms in a single flat list only.

The original SDA7 code and author attribution are retained; consult the SDA7 sources for original licensing and authorship details.

Author

Neil Bruce

Abraham Muñiz-Chicharro (modifications and extensions)

Version

1.0

Date

2026-04-05

11.12 mod_threshold.f90 File Reference

Utilities to compute threshold arrays from encounter complexes.

Modules

- module [mod_threshold](#)

Functions/Subroutines

- subroutine [mod_threshold::array_z_coord](#) (array, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, solute_crds)
Compute the Z-coordinate (along transformed z-axis) for each encounter.
- subroutine [mod_threshold::array_atoms_dist](#) (array, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, solute1↔_crds, solute2_crds)
Compute minimum inter-atomic distance between two solutes for each encounter.
- subroutine [mod_threshold::array_angle](#) (array, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, point1a, point1b, point2a, point2b, dimensions)
Compute angle between two vectors (defined by atom groups) per encounter.
- subroutine [mod_threshold::sort_array](#) (arr, encounter_indexes)
Sort an array in ascending order and reorder encounter indexes accordingly.

11.12.1 Detailed Description

Utilities to compute threshold arrays from encounter complexes.

Author

Abraham Muñoz-Chicharro

Version

1.0

Date

2026-04-05

11.13 read_input.f90 File Reference

Input readers for PDB and association/complexes files.

Modules

- module [read_input](#)

Functions/Subroutines

- subroutine [read_input::read_pdb](#) (pdb, filename, tot_atoms, tot_residues, tot_chains)
Read a PDB file into a `type_pdb_file` object.
- subroutine [read_input::read_assoc](#) (assoc, filename, tot_encounters)
Read an association/complexes file into a `type_assoc_file` object.

11.13.1 Detailed Description

Input readers for PDB and association/complexes files.

Author

Abraham Muñoz-Chicharro

Version

1.0

Date

2026-04-05

11.14 threshold.f90 File Reference

Threshold-based encounter selection from precomputed arrays.

Functions/Subroutines

- program [main](#)
- subroutine [parse_arguments](#) (p_array_input_filename, p_array_input_bool, p_complexes_filename, p_complexes_bool, p_datadist_filename, p_array_bool, p_output_complexes_filename, p_output_complexes_bool, p_nb_encounters, p_nb_encounters_bool, p_cutoff, p_cutoff_bool, p_help_bool)

Parse command-line arguments.

11.14.1 Detailed Description

Threshold-based encounter selection from precomputed arrays.

[threshold.f90](#) - Filter encounters using an input array generated by `make_data`.

Usage: `./threshold -help`

Reads a precomputed per-encounter array and the matching complexes file, applies a cutoff, and writes filtered array values and filtered complexes.

Author

Abraham Muniz-Chicharro

Version

1.0

Date

2026-06-07

11.14.2 Function/Subroutine Documentation

11.14.2.1 `main()`

program `main`

11.14.2.2 parse_arguments()

```
subroutine main::parse_arguments (
    character(len=128), intent(out) p_array_input_filename,
    logical, intent(out) p_array_input_bool,
    character(len=128), intent(out) p_complexes_filename,
    logical, intent(out) p_complexes_bool,
    character(len=128), intent(out) p_datadist_filename,
    logical, intent(out) p_array_bool,
    character(len=128), intent(out) p_output_complexes_filename,
    logical, intent(out) p_output_complexes_bool,
    integer, intent(out) p_nb_encounters,
    logical, intent(out) p_nb_encounters_bool,
    real(kind=8), intent(out) p_cutoff,
    logical, intent(out) p_cutoff_bool,
    logical, intent(out) p_help_bool )
```

Parse command-line arguments.

11.15 threshold.md File Reference

11.16 unit_test/README.md File Reference

11.17 unit_test/run_all_tests.sh File Reference

11.18 unit_test/test_array.f90 File Reference

Functions/Subroutines

- program [test_array](#)
Unit tests for array operations in the [mod_array](#) module.
- subroutine [test_write_read_array](#) (num_tests, num_passed, num_failed)
Unit test for write_array and read_array routines.

11.18.1 Function/Subroutine Documentation

11.18.1.1 test_array()

```
program test_array
```

Unit tests for array operations in the [mod_array](#) module.

Tests include:

- Write and read array operations
- Z-coordinate array sorting with encounter metadata reordering
- Minimum-distance array sorting with encounter metadata reordering
- Angle array sorting with encounter metadata reordering
- Low-level merge of sorted segments with metadata reordering

Author

Abraham Muñoz-Chicharro

Version

1.0

11.18.1.2 test_write_read_array()

```
subroutine test_array::test_write_read_array (
    integer, intent(inout) num_tests,
    integer, intent(inout) num_passed,
    integer, intent(inout) num_failed )
```

Unit test for write_array and read_array routines.

Author

Abraham

Version

1.0

Date

2026-04-06

Parameters

in, out	<i>num_tests</i>	Number of tests executed (incremented)
in, out	<i>num_passed</i>	Number of tests passed (incremented)
in, out	<i>num_failed</i>	Number of tests failed (incremented)

11.19 unit_test/test_assoc.f90 File Reference**Functions/Subroutines**

- program [test_assoc](#)
Unit tests for reading association/complexes files.
- subroutine [test_read_assoc_file](#) (num_tests, num_passed, num_failed)
Unit test for read_assoc file operations.

11.19.1 Function/Subroutine Documentation

11.19.1.1 test_assoc()

```
program test_assoc
```

Unit tests for reading association/complexes files.

Comprehensive test suite for reading and parsing association/complexes files in the [read_input](#) and [mod_assoc](#) modules including:

- **File reading:**
 - Valid assoc file reading
 - File existence checking
 - Error handling for missing files
 - Correct number of encounters parsed
- **Data structure allocation:**
 - Allocate assoc objects with correct sizes
 - Initialize translation vectors to zero
 - Initialize rotation vectors (rot1, rot2) to zero
 - Allocate lines array
- **File parsing:**
 - Skip comment lines (starting with #)
 - Count data lines correctly

Author

Abraham Muñoz-Chicharro

Version

1.0

- Extract center coordinates (xc1, xc2)
- Parse translation vectors
- Parse rotation vectors
- **Data integrity:**
 - Verify allocations match encounter count
 - Check header information is stored
 - Verify vector values are reasonable
 - Ensure no data corruption during read

Compile with: gfortran -o test_assoc [test_assoc.f90](#) ../src/read_input.f90 ../src/mod_assoc.f90

Author

Abraham Muñoz-Chicharro

Version

1.0

11.19.1.2 test_read_assoc_file()

```
subroutine test_assoc::test_read_assoc_file (
    integer, intent(inout) num_tests,
    integer, intent(inout) num_passed,
    integer, intent(inout) num_failed )
```

Unit test for read_assoc file operations.

Author

Abraham

Version

1.0

Date

2024-06-09

Parameters

in, out	<i>num_tests</i>	Number of tests executed (incremented)
in, out	<i>num_passed</i>	Number of tests passed (incremented)
in, out	<i>num_failed</i>	Number of tests failed (incremented)

11.20 unit_test/test_clustering.f90 File Reference

Unit tests for clustering algorithms in [mod_clust_algorithm](#).

Functions/Subroutines

- program [test_clustering](#)
- subroutine [test_clustering_statistics](#) (num_tests, num_passed, num_failed)
Test clustering statistics calculations (mean, stddev)

11.20.1 Detailed Description

Unit tests for clustering algorithms in [mod_clust_algorithm](#).

Tests include:

- Write cluster elements
- Basic clustering threshold calculation

- Mean linkage clustering

Note: Clustering now operates on sorted 1D arrays.

Compile with: gfortran -fopenmp -o test_clustering [test_clustering.f90](#) ../src/mod_clust_algorithm.f90 ../src/read_↵
input.f90

Author

Abraham Muniz-Chicharro

Version

1.0

Date

2026-04-05

11.20.2 Function/Subroutine Documentation

11.20.2.1 test_clustering()

```
program test_clustering
```

11.20.2.2 test_clustering_statistics()

```
subroutine test_clustering::test_clustering_statistics (
    integer, intent(inout) num_tests,
    integer, intent(inout) num_passed,
    integer, intent(inout) num_failed )
```

Test clustering statistics calculations (mean, stddev)

Parameters

in, out	<i>num_tests</i>	Number of tests run
in, out	<i>num_passed</i>	Number of tests passed
in, out	<i>num_failed</i>	Number of tests failed

11.21 unit_test/test_mathsf90 File Reference

Functions/Subroutines

- program [test_mathsf90](#)
Unit tests for mathematical operations in the maths module.
- subroutine [test_cross_product](#) (num_tests, num_passed, num_failed)
Unit test for cross product routine.

11.21.1 Function/Subroutine Documentation

11.21.1.1 test_cross_product()

```
subroutine test_maths::test_cross_product (
    integer, intent(inout) num_tests,
    integer, intent(inout) num_passed,
    integer, intent(inout) num_failed )
```

Unit test for cross product routine.

Author

Abraham

Version

1.0

Date

2024-06-09

Parameters

in, out	<i>num_tests</i>	Number of tests executed (incremented)
in, out	<i>num_passed</i>	Number of tests passed (incremented)
in, out	<i>num_failed</i>	Number of tests failed (incremented)

11.21.1.2 test_maths()

```
program test_maths
```

Unit tests for mathematical operations in the maths module.

Comprehensive test suite for all subroutines in the maths module including:

- **Cross product:**
 - Basis vectors ($i \times j = k$)
 - Reverse order (anticommutative property)
 - Self cross product ($v \times v = 0$)
 - Arbitrary vectors
- **Coordinate transformations (update_complex):**
 - Identity transformations
 - Pure translation
 - Only xc1 translation

- Only xc2 translation
- Only rotation (90 degrees)
- Combined transformations
- **3D angle calculations (vectors_angle_3D):**
 - Parallel vectors (0 degrees)
 - Perpendicular vectors (90 degrees)
 - Opposite vectors (180 degrees)
 - 45 degree angles
- **2D angle calculations (vectors_angle_2D):**
 - Parallel vectors in XY plane (0 degrees)
 - Perpendicular vectors in XY plane (90 degrees)
 - Opposite vectors in XY plane (180 degrees)
 - Zero magnitude vectors handling
- **Center of geometry (calculate_cog):**
 - Single point
 - Points at origin
 - Symmetric point distributions
 - General cases
- **Distance calculation (calculate_distance):**
 - Distance to self (0)
 - Unit distances
 - 3D diagonal distances
 - Arbitrary point pairs

Compile with: gfortran -o test_maths [test_maths.f90](#) ../src/maths.f90 ../src/mod_pdb.f90

Author

Abraham Muñiz-Chicharro

Version

1.0

11.22 unit_test/test_pdb.f90 File Reference

Functions/Subroutines

- program [test_pdb](#)
 - Unit tests for PDB operations in the [mod_pdb](#) module.*
- subroutine [test_pdb_allocation](#) (num_tests, num_passed, num_failed)
 - Unit test for PDB allocation and initialization routines.*

11.22.1 Function/Subroutine Documentation

11.22.1.1 test_pdb()

```
program test_pdb
```

Unit tests for PDB operations in the [mod_pdb](#) module.

Tests include:

- PDB type allocation and initialization
- Atom and residue management
- Residue structure handling
- Reading PDB files and filling the PDB data structure

Compile with: gfortran -o test_pdb [test_pdb.f90](#) [mod_pdb.f90](#)

Author

Abraham Muñoz-Chicharro

Version

1.0

11.22.1.2 test_pdb_allocation()

```
subroutine test_pdb::test_pdb_allocation (
    integer, intent(inout) num_tests,
    integer, intent(inout) num_passed,
    integer, intent(inout) num_failed )
```

Unit test for PDB allocation and initialization routines.

Author

Abraham

Version

1.0

Date

2024-06-09

Parameters

in, out	<i>num_tests</i>	Number of tests executed (incremented)
in, out	<i>num_passed</i>	Number of tests passed (incremented)
in, out	<i>num_failed</i>	Number of tests failed (incremented)

11.23 unit_test/test_read_input.f90 File Reference

Functions/Subroutines

- program [test_read_input](#)
Unit tests for the [read_input](#) module.
- subroutine [test_read_input_assoc](#) (num_tests, num_passed, num_failed)
Unit test for reading association files via [read_input](#) module.

11.23.1 Function/Subroutine Documentation

11.23.1.1 test_read_input()

```
program test_read_input
```

Unit tests for the [read_input](#) module.

Comprehensive test suite for reading PDB and association/complexes files using the [read_input](#) module wrapper functions including:

- **PDB file reading ([read_pdb](#)):**
 - Valid PDB file reading
 - File existence checking
 - Atom count verification
 - Residue count verification
 - Chain count verification
 - Proper structure allocation
 - Error handling for missing files
- **Association file reading ([read_assoc](#)):**
 - Valid assoc file reading
 - Encounter count verification
 - Structure allocation and initialization
 - Vector data integrity
 - Error handling for missing files

Author

Abraham Muñiz-Chicharro

Version

1.0

• Integration tests:

- Sequential reading of multiple files
- Proper cleanup between reads
- Memory management

Compile with: gfortran -o test_read_input [test_read_input.f90](#) ../src/read_input.f90 ../src/mod_pdb.f90 ../src/mod_
_assoc.f90

Author

Abraham Muñoz-Chicharro

Version

1.0

11.23.1.2 test_read_input_assoc()

```
subroutine test_read_input::test_read_input_assoc (  
    integer, intent(inout) num_tests,  
    integer, intent(inout) num_passed,  
    integer, intent(inout) num_failed )
```

Unit test for reading association files via [read_input](#) module.

Author

Abraham

Version

1.0

Date

2024-06-09

Parameters

in, out	<i>num_tests</i>	Number of tests executed (incremented)
in, out	<i>num_passed</i>	Number of tests passed (incremented)
in, out	<i>num_failed</i>	Number of tests failed (incremented)

11.24 unit_test/test_threshold.f90 File Reference

Functions/Subroutines

- program [test_threshold](#)
Unit tests for threshold operations in the [mod_threshold](#) module.
- subroutine [test_array_z_coord](#) (num_tests, num_passed, num_failed)
Unit test for array_z_coord routine.

11.24.1 Function/Subroutine Documentation

11.24.1.1 test_array_z_coord()

```
subroutine test_threshold::test_array_z_coord (  
    integer, intent(inout) num_tests,  
    integer, intent(inout) num_passed,  
    integer, intent(inout) num_failed )
```

Unit test for array_z_coord routine.

Author

Abraham

Version

1.0

Date

2024-06-09

Parameters

in, out	<i>num_tests</i>	Number of tests executed (incremented)
in, out	<i>num_passed</i>	Number of tests passed (incremented)
in, out	<i>num_failed</i>	Number of tests failed (incremented)

11.24.1.2 test_threshold()

```
program test_threshold
```

Unit tests for threshold operations in the [mod_threshold](#) module.

Tests include:

- Z-coordinate array calculation

- Minimum distance calculation between atoms
- Angle computation for different dimensions
- Array sorting with encounter index tracking

Compile with: gfortran -fopenmp -o test_threshold [test_threshold.f90](#) ../src/mod_threshold.f90 ../src/mathsf90
../src/mod_pdb.f90

Author

Abraham Muñiz-Chicharro

Version

1.0

Index

- allocate_assoc_object
 - mod_assoc, [24](#)
- allocate_pdb_object
 - mod_pdb, [28](#)
- array_angle
 - mod_array, [20](#)
 - mod_threshold, [30](#)
- array_atoms_dist
 - mod_array, [21](#)
 - mod_threshold, [31](#)
- array_z_coord
 - mod_array, [21](#)
 - mod_threshold, [31](#)
- atoms
 - mod_pdb::type_pdb_chain, [39](#)
 - mod_pdb::type_pdb_file, [41](#)
 - mod_pdb::type_pdb_residue, [42](#)
- beta
 - mod_pdb::type_pdb_atom, [37](#)
- calculate_cog
 - maths, [17](#)
- calculate_distance
 - maths, [18](#)
- chainid
 - mod_pdb::type_pdb_atom, [37](#)
 - mod_pdb::type_pdb_chain, [39](#)
 - mod_pdb::type_pdb_residue, [42](#)
- chains
 - mod_pdb::type_pdb_file, [41](#)
- clust.f90, [1](#), [43](#)
 - main, [44](#)
 - parse_arguments, [44](#)
- clust.md, [44](#)
- contact_map.f90, [3](#), [44](#)
 - main, [45](#)
 - print_help, [45](#)
- contact_map.md, [45](#)
- coord
 - mod_pdb::type_pdb_atom, [37](#)
- count_atoms
 - mod_pdb, [28](#)
- count_chains
 - mod_pdb, [28](#)
- count_residues
 - mod_pdb, [29](#)
- cross
 - maths, [18](#)
- fill_assoc_object
 - mod_assoc, [24](#)
- fill_pdb_object
 - mod_pdb, [29](#)
- Fortran Unit Tests (<tt>src/unit_test</tt>), [9](#)
- head
 - mod_assoc::type_assoc_file, [35](#)
- id
 - mod_pdb::type_pdb_atom, [37](#)
- lines
 - mod_assoc::type_assoc_file, [35](#)
- linkage_clustering
 - mod_clust_algorithm, [25](#)
- main
 - clust.f90, [44](#)
 - contact_map.f90, [45](#)
 - make_data.f90, [46](#)
 - threshold.f90, [54](#)
- make_data.f90, [5](#), [45](#)
 - main, [46](#)
 - parse_arguments, [46](#)
- make_data.md, [48](#)
- maths, [17](#)
 - calculate_cog, [17](#)
 - calculate_distance, [18](#)
 - cross, [18](#)
 - update_complex, [18](#)
 - vectors_angle_2d, [19](#)
 - vectors_angle_3d, [19](#)
- maths.f90, [48](#)
- merge_sorted_segments
 - mod_array, [22](#)
- mod_array, [20](#)
 - array_angle, [20](#)
 - array_atoms_dist, [21](#)
 - array_z_coord, [21](#)
 - merge_sorted_segments, [22](#)
 - read_array, [22](#)
 - write_array, [23](#)
- mod_array.f90, [49](#)
- mod_assoc, [23](#)
 - allocate_assoc_object, [24](#)
 - fill_assoc_object, [24](#)
 - size_assoc, [24](#)
 - write_complexes, [25](#)
- mod_assoc.f90, [49](#)

- mod_assoc::type_assoc_file, 35
 - head, 35
 - lines, 35
 - nlines, 36
 - rot1, 36
 - rot2, 36
 - trans_vector, 36
 - xc1, 36
 - xc2, 36
- mod_clust_algorithm, 25
 - linkage_clustering, 25
 - write_clust_info, 26
 - write_cluster_complexes, 26
 - write_cluster_elements, 26
- mod_clust_algorithm.f90, 50
- mod_pdb, 27
 - allocate_pdb_object, 28
 - count_atoms, 28
 - count_chains, 28
 - count_residues, 29
 - fill_pdb_object, 29
- mod_pdb.f90, 51
- mod_pdb::type_pdb_atom, 37
 - beta, 37
 - chainid, 37
 - coord, 37
 - id, 37
 - name, 37
 - occ, 37
 - record, 38
 - resid, 38
 - resname, 38
- mod_pdb::type_pdb_chain, 38
 - atoms, 39
 - chainid, 39
 - natoms, 39
 - nresidues, 39
 - residues, 39
- mod_pdb::type_pdb_file, 40
 - atoms, 41
 - chains, 41
 - natoms, 41
 - nchains, 41
 - nresidues, 41
 - residues, 41
- mod_pdb::type_pdb_residue, 41
 - atoms, 42
 - chainid, 42
 - natoms, 42
 - resid, 42
 - resname, 42
- mod_threshold, 30
 - array_angle, 30
 - array_atoms_dist, 31
 - array_z_coord, 31
 - sort_array, 32
- mod_threshold.f90, 52
- name
 - mod_pdb::type_pdb_atom, 37
- natoms
 - mod_pdb::type_pdb_chain, 39
 - mod_pdb::type_pdb_file, 41
 - mod_pdb::type_pdb_residue, 42
- nchains
 - mod_pdb::type_pdb_file, 41
- nlines
 - mod_assoc::type_assoc_file, 36
- nresidues
 - mod_pdb::type_pdb_chain, 39
 - mod_pdb::type_pdb_file, 41
- occ
 - mod_pdb::type_pdb_atom, 37
- parse_arguments
 - clust.f90, 44
 - make_data.f90, 46
 - threshold.f90, 54
- print_help
 - contact_map.f90, 45
- read_array
 - mod_array, 22
- read_assoc
 - read_input, 32
- read_input, 32
 - read_assoc, 32
 - read_pdb, 33
- read_input.f90, 53
- read_pdb
 - read_input, 33
- record
 - mod_pdb::type_pdb_atom, 38
- resid
 - mod_pdb::type_pdb_atom, 38
 - mod_pdb::type_pdb_residue, 42
- residues
 - mod_pdb::type_pdb_chain, 39
 - mod_pdb::type_pdb_file, 41
- resname
 - mod_pdb::type_pdb_atom, 38
 - mod_pdb::type_pdb_residue, 42
- rot1
 - mod_assoc::type_assoc_file, 36
- rot2
 - mod_assoc::type_assoc_file, 36
- size_assoc
 - mod_assoc, 24
- sort_array
 - mod_threshold, 32
- test_array
 - test_array.f90, 55
- test_array.f90
 - test_array, 55
 - test_write_read_array, 56

- test_array_z_coord
 - test_threshold.f90, [65](#)
- test_assoc
 - test_assoc.f90, [57](#)
- test_assoc.f90
 - test_assoc, [57](#)
 - test_read_assoc_file, [57](#)
- test_clustering
 - test_clustering.f90, [59](#)
- test_clustering.f90
 - test_clustering, [59](#)
 - test_clustering_statistics, [59](#)
- test_clustering_statistics
 - test_clustering.f90, [59](#)
- test_cross_product
 - test_maths.f90, [60](#)
- test_maths
 - test_maths.f90, [60](#)
- test_maths.f90
 - test_cross_product, [60](#)
 - test_maths, [60](#)
- test_pdb
 - test_pdb.f90, [62](#)
- test_pdb.f90
 - test_pdb, [62](#)
 - test_pdb_allocation, [62](#)
- test_pdb_allocation
 - test_pdb.f90, [62](#)
- test_read_assoc_file
 - test_assoc.f90, [57](#)
- test_read_input
 - test_read_input.f90, [63](#)
- test_read_input.f90
 - test_read_input, [63](#)
 - test_read_input_assoc, [64](#)
- test_read_input_assoc
 - test_read_input.f90, [64](#)
- test_threshold
 - test_threshold.f90, [65](#)
- test_threshold.f90
 - test_array_z_coord, [65](#)
 - test_threshold, [65](#)
- test_write_read_array
 - test_array.f90, [56](#)
- threshold.f90, [7](#), [54](#)
 - main, [54](#)
 - parse_arguments, [54](#)
- threshold.md, [55](#)
- trans_vector
 - mod_assoc::type_assoc_file, [36](#)
- unit_test/README.md, [55](#)
- unit_test/run_all_tests.sh, [55](#)
- unit_test/test_array.f90, [55](#)
- unit_test/test_assoc.f90, [56](#)
- unit_test/test_clustering.f90, [58](#)
- unit_test/test_maths.f90, [59](#)
- unit_test/test_pdb.f90, [61](#)
- unit_test/test_read_input.f90, [63](#)
- unit_test/test_threshold.f90, [65](#)
- update_complex
 - maths, [18](#)
- vectors_angle_2d
 - maths, [19](#)
- vectors_angle_3d
 - maths, [19](#)
- write_array
 - mod_array, [23](#)
- write_clust_info
 - mod_clust_algorithm, [26](#)
- write_cluster_complexes
 - mod_clust_algorithm, [26](#)
- write_cluster_elements
 - mod_clust_algorithm, [26](#)
- write_complexes
 - mod_assoc, [25](#)
- xc1
 - mod_assoc::type_assoc_file, [36](#)
- xc2
 - mod_assoc::type_assoc_file, [36](#)